

Módulo de Perfis, Acessos e Filtro

Autor: Anderson Felipe Dias Domingos

Data de Atualização: 16/07/2026

Status: Concluído

1. Visão Geral e Objetivo

O módulo de perfis, acessos e filtros tem como objetivo estruturar a base de autenticação e a segurança de rotas da Plataforma de Gestão de Estágios em Psicologia (SEP).

Este módulo define a modelagem de usuário customizado herdando de `AbstractUser`, implementando um fluxo de login baseado estritamente em e-mail (removendo o campo nativo de username) e centralizando a hierarquia de cargos do sistema. Além disso, estabelece as barreiras de proteção de rotas para Views Baseadas em Função (FBVs) e Views Baseadas em Classe (CBVs) através de decoradores e mixins customizados, garantindo que a privacidade dos dados seja mantida na camada de banco de dados via QuerySets Dinâmicos.

2. Pesquisa e Fundamentação (O que foi estudado)

Documente aqui os resultados das suas pesquisas antes e durante a implementação.

- **Conceitos Chave:** Abstract User vs Abstract Base User, Functions Based Views(FBVs) x Class Based Views(CBVs), `models.TextChoices`, decorators, mixins e QuerySets Dinâmicos
- **Bibliotecas e Ferramentas:** Django.
- **Decisões Arquiteturais (Prós e Contras):**
 - Para a criação do modelo de usuário personalizado, a herança de **AbstractUser** é a mais indicada por já trazer a estrutura padrão do Django (como e-mail e validações nativas), garantindo agilidade no desenvolvimento. A alternativa, **AbstractBaseUser**, oferece apenas a criptografia de senha e destina-se a sistemas com dados de autenticação totalmente fora do padrão comum.
 - No desenvolvimento das telas, a escolha entre **FBV (Views Baseadas em Função)** e **CBV (Views Baseadas em Classe)** ocorre a cada nova rota. As FBVs são funções sequenciais e diretas, ideais para fluxos de alta customização. Já as CBVs utilizam classes e herança para eliminar códigos repetitivos em operações clássicas de banco de dados (CRUD).
 - A definição dos cargos (papéis) no sistema é feita com o enumerador **`models.TextChoices`**, onde cada opção é declarada internamente no modelo de usuário com uma constante, um valor de banco e um rótulo amigável, sendo então associada a

um campo de texto convencional (CharField) por meio do parâmetro de escolhas (choices).

- O controle de acesso e a segurança das páginas são garantidos por barreiras que impedem visualizações não autorizadas. Os **Decorators** envolvem e protegem diretamente as funções (FBVs), enquanto os **Mixins** utilizam herança múltipla para blindar o acesso às classes (CBVs).
- Por fim, a privacidade das informações é assegurada por **QuerySets Dinâmicos**, que extraem as credenciais do usuário autenticado diretamente da requisição ativa para filtrar as consultas ao banco de dados, garantindo que o cliente logado visualize estritamente as suas próprias tarefas.

3. Guia de Configuração e Instalação (Setup)

Passo a passo exato de como um outro desenvolvedor da equipe faz essa feature funcionar na máquina dele.

- **Repositório:** <https://github.com/Felipe-dmgs/Teste>
- **Dependências:** pip install django
- **Comandos Específicos:**
 - python manage.py makemigrations users
 - python manage.py migrate
 - python manage.py createsuperuser
 - python manage.py runserver

4. Implementação e Casos de Uso (Como Usar)

4.1. Descrição dos Arquivos

Como primeiro Objetivo temos a criação de usuários customizados e para fazer isso também precisamos de um UserManager customizado já que o UserManager padrão do django não consegue manejar um usuário customizado.

managers.py:

Este código apresenta o desenvolvimento de um **UserManager** customizado. Nas primeiras linhas, importamos o BaseUserManager do Django e o gettext_lazy. O BaseUserManager é herdado por CustomUserManager para fornecer os métodos utilitários padrões de manipulação de usuários no banco de dados. O gettext_lazy é utilizado para garantir a tradução tardia das strings de erro, traduzindo-as apenas quando forem renderizadas para o usuário final, e não na inicialização do servidor.

Em seguida, criamos a classe CustomUserManager contendo duas funções essenciais: create_user (criação de usuário comum) e create_superuser (criação de superusuário).

Na função `create_user`, alteramos o identificador padrão de login do Django de `username` para `e-mail`. Para isso, definimos o e-mail como um parâmetro obrigatório (disparando um erro caso não seja fornecido) e aplicamos o método `normalize_email`, que converte o domínio do e-mail (a parte após o '@') para letras minúsculas. O usuário é então instanciado com os campos adicionais (`extra_fields`), sua senha é criptografada de forma segura através do método `set_password`, e o registro é salvo no banco de dados e é retornado.

Na função `create_superuser`, seguimos o mesmo fluxo de criação, porém definindo por padrão as flags administrativas `is_staff`, `is_superuser` e `is_active` como `True`. O método valida se essas permissões foram configuradas corretamente e, caso positivo, invoca a função `create_user` para registrar o superusuário de forma segura.

Models.py:

Nas primeiras linhas do arquivo, realizamos as importações fundamentais do Django: o módulo `models` para a definição de tabelas, `AbstractUser` para estender o modelo de autenticação padrão, `gettext_lazy` para a tradução tardia de rótulos, `settings` para referenciar de forma segura o modelo de usuário configurado no projeto, e o `CustomUserManager` que gerenciará as operações de criação desse novo usuário.

A classe `CustomUser` herda de `AbstractUser` para manter a robustez das validações nativas do Django, aplicando modificações para adaptar o sistema. Primeiramente, removemos o campo padrão de usuário definindo `username = None` e estabelecemos o campo `email` como o identificador único e obrigatório (`unique=True`). Consequentemente, configuramos `USERNAME_FIELD = "email"`, definindo-o como a credencial de login do sistema, e limpamos a lista de campos extras obrigatórios (`REQUIRED_FIELDS = []`).

Para a definição de perfis de acesso, implementamos a classe interna `Role` herdando de `models.TextChoices`. Esse enumerador (`Enum`) centraliza de forma segura e legível os cargos disponíveis no sistema: `SUPERADMIN`, `SUPERVISOR`, `PROFESSOR`, `ADMIN`, `ALUNO` e `PACIENTE`. Esse mapeamento é integrado ao banco de dados através do campo `role` (um `CharField` com tamanho máximo de 2 caracteres), que consome as opções declaradas e define `Role.ALUNO` como papel padrão para novos cadastros. Por fim, vinculamos o gerenciador customizado através de `objects = CustomUserManager()` e definimos que a representação textual do objeto (`__str__`) retornará o e-mail do usuário.

Na sequência, definimos o modelo `Tarefa`, que representa as atividades cadastradas no sistema. Ele possui um relacionamento de chave estrangeira (`ForeignKey`) com o modelo de usuário do projeto utilizando a constante `settings.AUTH_USER_MODEL` — prática recomendada para evitar importações circulares. Definimos a exclusão em cascata (`on_delete=models.CASCADE`), o que significa que se um usuário for deletado, suas tarefas também serão excluídas do banco de dados. O modelo contém campos de texto para título e descrição (sendo este opcional), um booleano de controle concluída e um campo de data criada_em que armazena automaticamente o momento de criação do registro.

Decorators.py:

Primeiro, realizamos as importações necessárias: o `HttpResponseForbidden` do Django, utilizado para retornar uma resposta HTTP de acesso negado (código de status 403), e o `wraps` do módulo nativo `functools`, essencial para preservar os metadados originais da view (como nome e documentação da função) ao ser envelopada pelo decorator.

A função `role_required` é um decorator customizado e parametrizável que aceita uma lista de papéis permitidos (`allowed_roles`). A sua estrutura é composta por três camadas de funções devido à necessidade de receber esses parâmetros externos:

1. A função externa `role_required` recebe e armazena os papéis autorizados a acessar a view.
2. A função intermediária `decorator` recebe a função de view original (`view_func`) que será protegida.
3. A função interna `_wrapped_view` (envelopada com `@wraps`) intercepta a requisição ativa (`request`) para aplicar duas regras de validação de segurança antes de permitir a execução da view:
 - Primeiro, verifica se o usuário está autenticado através de `request.user.is_authenticated`. Caso não esteja, a requisição é abortada imediatamente com uma resposta de erro `HttpResponseForbidden`.
 - Segundo, avalia se o papel do usuário autenticado (`request.user.role`) está presente na lista de cargos permitidos (`allowed_roles`).

Se ambas as validações forem bem-sucedidas, a execução prossegue normalmente retornando a view original com seus respectivos argumentos (`*args` e `**kwargs`). Caso contrário, o decorator bloqueia o acesso e retorna uma resposta do tipo `HttpResponseForbidden` informando que o usuário não possui a permissão necessária.

Mixins.py:

No início do arquivo, realizamos as importações necessárias para a construção da segurança baseada em classes: a exceção `PermissionDenied` do Django, utilizada para disparar um erro HTTP 403 (Acesso Proibido) de forma padronizada pelo framework, e a classe `AccessMixin` do módulo de autenticação do Django, que serve como classe base para a criação de mixins personalizados de controle de acesso.

A classe `RoleRequiredMixin` herda de `AccessMixin` e foi desenvolvida para proteger Class-Based Views (CBVs). Ela define o atributo de classe `allowed_roles` inicializado como uma lista vazia, o qual deve ser sobrescrito na view que herda este mixin com os cargos permitidos para aquele endpoint.

O controle de fluxo ocorre por meio da sobrescrita do método `dispatch`, que é o ponto de entrada principal de qualquer CBV no Django (responsável por receber a requisição e direcioná-la para o método HTTP correspondente, como GET ou POST). O método realiza a

validação de segurança em duas etapas:

1. Primeiramente, verifica se o usuário está autenticado (`request.user.is_authenticated`). Caso não esteja, invoca o método herdado `self.handle_no_permission()`, que redireciona automaticamente o usuário anônimo para a tela de login configurada no sistema.
2. Em seguida, caso o usuário esteja logado, avalia se o seu papel (`request.user.role`) consta na lista de cargos permitidos (`self.allowed_roles`).

Se a validação do papel for bem-sucedida, o mixin concede o acesso invocando o comportamento original da view através de `super().dispatch(request, *args, **kwargs)`. Caso o usuário esteja autenticado mas não possua o cargo exigido, o sistema interrompe a execução e dispara a exceção `PermissionDenied`, exibindo a mensagem de restrição de acesso configurada.

4.2. Exemplos Práticos de Uso

AbstractUser e Roles:

Class CustomUser(AbstractUser):

...

class Role(models.TextChoices):

SUPERADMIN = 'SA',

SUPERVISOR = 'SV'

PROFESSOR = 'PR'

ADMIN: 'AD'

ALUNO: 'AL'

PACIENTE: 'PA'

role = models.CharField(

max_length = 2,

choices = Role.choices,

default = Role.ALUNO

)

Aplicação do QuerySet dinâmico e Mixins em Class Based Views:

```
class PaineSupervisorView(RoleRequiredMixin, ListView):

    model = CustomUser

    template_name = 'usuarios/paine_supervisor.html'

    context_object_name = 'usuarios_listados'

    allowed_roles = [CustomUser.Role.SUPERVISOR]

    def get_queryset(self):

        return CustomUser.objects.filter(

            role__in = [CustomUser.Role.ALUNO, CustomUser.Role.PROFESSOR ]

        )
```

Aplicação de QuerySet Dinâmico e Decorators em Function Based Views:

```
@role_required(allowed_roles=[CustomUser.Role.PROFESSOR])
def paine_professor_fbv(request):

    alunos = CustomUser.objects.filter(role=CustomUser.Role.ALUNO)

    context = {

        'alunos': alunos

    }

    return render(request, 'usuarios/paine_professor.html', context)
```

5. Testes e Validação

- **Validação de Controle de Acesso (Erro 403):** Realizei o login no sistema utilizando credenciais de um usuário com o perfil Aluno e tentei acessar a URL do painel restrito aos Supervisores (/paine-supervisor/). O sistema interceptou a requisição com sucesso e retornou o erro **HTTP 403 (Forbidden)**. Isso comprova que o RoleRequiredMixin e o decorator @role_required estão funcionando perfeitamente, impedindo o acesso de

usuários autenticados que não possuem o cargo exigido para a rota.

- **Validação de Integridade do Banco de Dados (Campos Obrigatórios):** Tentei registrar um novo usuário através do formulário de cadastro deixando o campo CPF em branco. O sistema bloqueou a ação no momento da submissão. Isso valida que o CustomUserCreationForm está respeitando as restrições do models.py (unique=True e a ausência de null=True), barrando dados incompletos antes mesmo de chegarem ao banco de dados e evitando erros de integridade (IntegrityError).
- **Validação de Privacidade de Dados (QuerySets Dinâmicos):** Acessei o painel de tarefas logado como um usuário do tipo Aluno. Ao renderizar a tela, a lista exibiu estritamente as tarefas vinculadas a este usuário em específico. Esse teste confirma que a sobrescrita do método get_queryset() utilizando filter(usuario=self.request.user) está correta, garantindo o isolamento dos dados e impedindo que um aluno visualize informações de outros usuários.
- **Validação de Filtros Múltiplos (Painel do Supervisor):** Logado como Supervisor, acessei a visão geral do sistema. A lista renderizada exibiu exclusivamente os usuários que possuem as roles de Aluno e Professor (e Administrativo, se aplicável), ocultando corretamente outros Supervisores e SuperAdmins. Isso comprova a eficácia da consulta otimizada utilizando role__in=[...] no QuerySet, filtrando e retornando os dados corretos em uma única transação no banco.

6. Referências e Links Úteis

Liste todos os links de documentações oficiais, tutoriais e artigos que você usou para estudar e implementar a feature.

- [Documentação Oficial](#)
- [CBVs x FBVs](#)
- [Custom User - AbstractUser x AbstractBaseUser](#)
- [Mixins](#)
- [decorators - Documentação Oficial](#)
- [decorators](#)